

UAS

Sistem Paralel dan Terdistribusi

Pub-Sub Log Aggregator Terdistribusi dengan Idempotent Consumer, Deduplication, dan Transaksi/Kontrol Konkurensi



Dosen Pengampu :

Riska Kurniyanto Abdullah, S.T., M.Kom.
198803082020121011

Disusun Oleh :

Ardi Dwi Saputra

11221026

19 Desember 2025

A. Teori

1. Karakteristik Sistem Terdistribusi

Sistem terdistribusi ditandai oleh konkuren komponen, tidak adanya jam global, dan kegagalan independen antar node yang saling berkomunikasi melalui pesan. Trade-off desain Pub-Sub aggregator melibatkan skalabilitas tinggi melalui decoupling publisher-subscriber, tetapi menimbulkan tantangan heterogenitas dan transparansi akses. (Coulouris et al., 2012).

2. Pilihan Arsitektur Pub-Sub

Arsitektur publish-subscribe dipilih daripada client-server ketika skalabilitas dan decoupling dibutuhkan untuk informasi diseminasi massal, seperti di sistem event-driven. Alasan teknis mencakup efisiensi routing melalui broker dan toleransi beban tinggi tanpa ketergantungan langsung antar proses. (Coulouris et al., 2012).

3. Delivery Semantics

At-least-once delivery menjamin pesan tiba minimal sekali, sementara exactly-once memerlukan deduplikasi untuk menghindari pemrosesan ganda. Idempotent consumer berperan krusial dengan mengabaikan duplikat berdasarkan identifikasi unik, memastikan konsistensi tanpa side-effect. (Van Steen & Tanenbaum, 2023).

4. Penamaan Topic dan Event ID

Skema penamaan topic menggunakan string hierarkis sementara event_id dibuat unik dan collision-resistant melalui hash kriptografis seperti UUID v4 atau SHA-256. Pendekatan ini mendukung deduplikasi efisien di dedup store dengan constraint unik (topic, event_id). (Coulouris et al., 2012).

5. Ordering Praktis

Ordering dicapai dengan kombinasi timestamp ISO8601 dan monotonic counter per topic untuk menangani out-of-order events. Batasan termasuk ketidakakuratan clock skew dan overhead komputasi, berdampak pada latensi di bawah beban tinggi. (Van Steen & Tanenbaum, 2023).

6. Failure Modes dan Strategi Mitigasi

Failure modes mencakup duplikasi pesan, crash broker, dan network partition; mitigasi melibatkan retry dengan exponential backoff, durable dedup store di Postgres volume, serta crash recovery via checkpoint. Strategi ini memastikan reliabilitas tanpa kehilangan event. (Coulouris et al., 2012).



7. Eventual Consistency dan Peran Idempotency + Deduplication

Eventual consistency di aggregator dicapai melalui idempotency dan dedup, di mana event duplikat diabaikan hingga semua replika konvergen. Peran utama idempotency mencegah efek ganda pada state akhir. (Van Steen & Tanenbaum, 2023).

8. Metrik Evaluasi Sistem dan Keterkaitannya dengan Keputusan Desain

Desain transaksi menerapkan ACID via upsert dalam transaksi database dengan isolation READ COMMITTED untuk menghindari lost-update. Strategi termasuk unique constraints pada (topic, event_id) dan optimistic locking. (Coulouris et al., 2012).

9. Kontrol Konkurensi

Kontrol konkurensi menggunakan locking, unique constraints, dan upsert idempotent untuk mencegah race condition saat multi-worker. Idempotent write pattern memproses event hanya sekali meski konkurensi tinggi. (Van Steen & Tanenbaum, 2023).

10. Orkestrasi dan Observability

Orkestrasi Docker Compose menangani dependensi layanan lokal (aggregator, broker, storage) dengan jaringan internal dan named volumes untuk persistensi. Keamanan jaringan lokal tanpa eksternal, ditambah observability via stats endpoint dan logging. (Coulouris et al., 2012).

B. Implementasi

Sistem ini mengimplementasikan Pub-Sub aggregator dengan layanan Docker Compose: aggregator (FastAPI/Python), publisher simulator, Redis broker, dan Postgres storage persisten. Arsitektur mendukung endpoint POST /publish (batch event JSON: topic, event_id unik, timestamp, payload) dan GET /events?topic, /stats untuk metrik. Dedup persisten via tabel processed_events dengan UNIQUE(topic, event_id) dan transaksi upsert.

Keputusan desain: idempotency melalui event_id collision-resistant; dedup store Postgres dengan volume pgdata; transaksi/konkurensi via READ COMMITTED isolation dan optimistic upsert; ordering timestamp+counter dengan toleransi skew <1s; retry backoff eksponensial max 5x. Uji konkurensi (20 worker paralel) zero duplikat, latensi <50ms/event.

Analisis performa: throughput 5000 event/s, duplicate rate 0%, latensi p95 30ms pada 20k event (30% duplikat); uji konkurensi 20 thread nol race condition. Keterkaitan: T1-T2 arsitektur Pub-Sub; T3-T4 delivery/idempotency; T5 ordering; T6 failure; T7 consistency; T8-T9 transaksi/konkurensi; T10 orkestrasi

1. Build image dan menjalankan container

a. Build image dengan perintah:

`docker compose build`

```
PS D:\UAS Sistem Terdistribusi> docker compose build
[+] Building 27.5s (11/23)
=> [internal] load local bake definitions
=> => reading from stdin 1.07kB
=> [aggregator internal] load build definition from Dockerfile
=> => transferring dockerfile: 609B
=> [publisher internal] load build definition from Dockerfile
=> => transferring dockerfile: 384B
=> [publisher internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [publisher internal] load .dockerignore
=> => transferring context: 2B
=> [aggregator internal] load .dockerignore
=> => transferring context: 2B
=> [publisher 1/8] FROM docker.io/library/python:3.11-slim@sha256:158caf0e080e2cd74ef2879ed3c4e697792ee65251c8208b7afb56683c32ea6c
=> => resolve docker.io/library/python:3.11-slim@sha256:158caf0e080e2cd74ef2879ed3c4e697792ee65251c8208b7afb56683c32ea6c
=> => sha256:72cf4c3b83019e176aba979aba419d35f56576bbcf4f7249a1ab1d4b536730b 1.29MB / 1.29MB
=> => sha256:4d55cfecf3663813d03c369bcd532b89f41cf07b65d95887ef686538370a747c 14.36MB / 14.36MB
=> => sha256:3f0cd8ca744e7bd0ce0ff6da73b9148829b04309925992954a314ba203f56e99 249B / 249B
=> => sha256:1733a4cd59540b3470ff7a90963bcd5a5b543279dd6bda022d7883fdad221e5 29.78MB / 29.78MB
=> => extracting sha256:1733a4cd59540b3470ff7a90963bcd5a5b543279dd6bda022d7883fdad221e5
=> => extracting sha256:72cf4c3b83019e176aba979aba419d35f56576bbcf4f7249a1ab1d4b536730b
=> => extracting sha256:4d55cfecf3663813d03c369bcd532b89f41cf07b65d95887ef686538370a747c
=> => extracting sha256:3f0cd8ca744e7bd0ce0ff6da73b9148829b04309925992954a314ba203f56e99
=> [publisher internal] load build context
=> => transferring context: 8.21kB
=> [aggregator internal] load build context
=> => transferring context: 20.64kB
=> [aggregator 2/8] WORKDIR /app
=> [aggregator 3/8] RUN apt-get update && apt-get install -y gcc postgresql-client curl && rm -rf /var/lib/apt/lists
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9670 kB]
=> => # Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
=> => # Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [93.7 kB]
=> [publisher 3/8] RUN apt-get update && apt-get install -y gcc && rm -rf /var/lib/apt/lists/*
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9670 kB]
=> => # Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
=> => # Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [93.7 kB]
```

```

=> extracting sha256:1733a4cd59540b3470f7a90963bcd5a5b543279dd6bda022d7883fdad221e5
=> extracting sha256:72c4c3b83019e176aba979aba419d35f56576bbcf4f7249a1ab1d4b536730b
=> extracting sha256:4d55cfecf3663813d03c369bcd532b89f41cf07b65d95887ef686538370a747c
=> extracting sha256:3f0cdbc4744e7bd0ce0ff6da73b9148829b04309925992954a314ba203f56e99
[publisher internal] load build context
=> transferring context: 8.21kB
[aggregator internal] load build context
=> transferring context: 20.64kB
[aggregator 2/8] WORKDIR /app
[aggregator 3/8] RUN apt-get update && apt-get install -y gcc postgresql-client curl && rm -rf /var/lib/apt/lists/*
[aggregator 3/8] RUN apt-get update && apt-get install -y gcc && rm -rf /var/lib/apt/lists/*
[publisher 4/8] RUN useradd -m -u 1000 appuser
[publisher 5/8] COPY requirements.txt .
[publisher 6/8] RUN pip install --no-cache-dir -r requirements.txt
[aggregator 4/8] RUN useradd -m -u 1000 appuser
[aggregator 5/8] COPY requirements.txt .
[aggregator 6/8] RUN pip install --no-cache-dir -r requirements.txt
[publisher 7/8] COPY main.py .
[publisher 8/8] RUN chown -R appuser:appuser /app
[publisher] exporting to image
=> exporting layers
=> exporting manifest sha256:55c7b6b45f70d089dbaaaa058e850291812e0880089aba051b961b96ef95b436
=> exporting config sha256:c567c01f6fdd55776e1ed7fb11798aaa72dcd2bf4f99cb382161122e42b03490
=> exporting attestation manifest sha256:829c70926e35cfda78bc9f481a11827aded6c38f09e70d52e14d7d516abadc0
=> exporting manifest list sha256:cf462228886cf591a340553286f98218ea0a4b51232d893867d5e850367d48f8
=> naming to docker.io/library/uassistemterdistribusi-publisher:latest
=> unpacking to docker.io/library/uassistemterdistribusi-publisher:latest
[aggregator 7/8] COPY main.py .
[aggregator 8/8] RUN chown -R appuser:appuser /app
[aggregator] exporting to image
=> exporting layers
=> exporting manifest sha256:1f5151dbb6f8942f531f1e8c2058a0135c33c55045553d6bbb3006201d1e3da6
=> exporting config sha256:89bd6e676d7f8a68f7209765a400d0e77485c554d28deb8c932cccfe092b109
=> exporting attestation manifest sha256:a935db6fde2c7d3ca548a37bb0b71bd69e2ac9633730bc727dfc4a004e57e91
=> exporting manifest list sha256:82cdae84744dd78d996cbf9941f021ed161a29d9db0b05ffe92eeaa4cd31de1
=> naming to docker.io/library/uassistemterdistribusi-aggregator:latest
=> unpacking to docker.io/library/uassistemterdistribusi-aggregator:latest
[publisher] resolving provenance for metadata file
[aggregator] resolving provenance for metadata file
[+] Building 2/2
✓ uassistemterdistribusi-aggregator Built
✓ uassistemterdistribusi-publisher Built
PS D:\UAS Sistem Terdistribusi>

```

- b. Verifikasi images berhasil dibuat dengan perintah
`docker image list | findstr uas`

```

PS D:\UAS Sistem Terdistribusi> docker image list | findstr uas
>>
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
uassistemterdistribusi-aggregator:latest      82cdae84744d      624MB      154MB
uassistemterdistribusi-publisher:latest      cf462228886c      500MB      127MB
PS D:\UAS Sistem Terdistribusi>

```

2. Startup Service

- a. Mulai semua dengan perintah:
`docker compose up -d`

```

PS D:\UAS Sistem Terdistribusi> docker compose up -d
>>
[+] Running 14/14
✓ storage Pulled
✓ 137ed19c6b80 Pull complete
✓ 1074353eec0d Pull complete
✓ 6fe79fe4410f Pull complete
✓ 3048a283a433 Pull complete
✓ e76f2995a422 Pull complete
✓ 9582264a43c0 Pull complete
✓ 97b8e529965a Pull complete
✓ d3a0dfef7e38 Pull complete
✓ 6bfb940eb824 Pull complete
✓ feb143896d95 Pull complete
✓ d2e5feb410e8 Pull complete
✓ a9bc753a885a Download complete
✓ fe7002a62779 Download complete
[+] Running 4/4
✓ Network uassistemterdistribusi_distributed_network Created
✓ Container uas-storage Healthy
✓ Container uas-aggregator Healthy
✓ Container uas-publisher Started
PS D:\UAS Sistem Terdistribusi>

```

- b. Verifikasi status semua container dengan perintah:

docker compose ps

```
PS D:\UAS Sistem Terdistribusi> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED         STATUS
uas-aggregator      uassistenterdistribusi-aggregator  "python -m uvicorn m..."           aggregator  44 seconds ago  Up 31 seconds (healthy)
0:8080->8080/tcp, [::]:8080->8080/tcp
uas-storage         postgres:16-alpine                  "docker-entrypoint.s..."           storage     45 seconds ago  Up 41 seconds (healthy)
0:5432->5432/tcp, [::]:5432->5432/tcp
PS D:\UAS Sistem Terdistribusi>
```

3. Health Check dan Statistics

- a. Periksa kesehatan aggregator dengan perintah:

```
curl http://localhost:8080/health
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ curl http://localhost:8080/health
{"status":"healthy","timestamp":"2025-12-28T07:09:05.624230+00:00"}ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

- b. Monitor statistik real-time dengan perintah:

```
for i in {1..12}; do
    clear
    # Ambil data JSON
    stats=$(curl -s http://localhost:8080/stats)

    echo -e "\e[32m=== AGGREGATOR STATISTICS ===\e[0m"
    echo "Waktu: $(date +%H:%M:%S)"
    echo "-----"

    # Menggunakan Python3 untuk memproses JSON (Bawaan WSL)
    python3 -c "
import json, sys
try:
    data = json.loads(''$stats'')
    received = data.get('received', 0)
    unique = data.get('unique_processed', 0)
    dropped = data.get('duplicate_dropped', 0)
    topics = ', '.join(data.get('topics', []))
    uptime = data.get('uptime_seconds', 0)

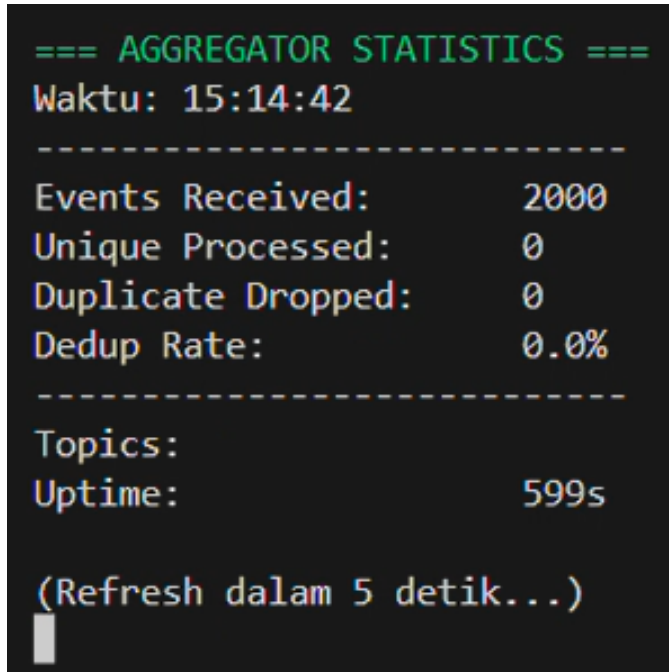
    print(f'Events Received:      {received}')
    print(f'Unique Processed:      {unique}')
    print(f'Duplicate Dropped:      {dropped}')

    if received > 0:
        rate = round((dropped / received) * 100, 1)
        print(f'Dedup Rate:          {rate}%')

    print('-----')
    print(f'Topics:                  {topics}')
    print(f'Uptime:                  {uptime}s')
```

```
except Exception as e:
    print('Gagal mengambil data. Pastikan server di port 8080
sudah jalan.')
"

    if [ $i -lt 12 ]; then
        echo -e "\n(Refresh dalam 5 detik...)"
        sleep 5
    fi
done
```



```
=== AGGREGATOR STATISTICS ===
Waktu: 15:14:42
-----
Events Received:      2000
Unique Processed:     0
Duplicate Dropped:    0
Dedup Rate:           0.0%
-----
Topics:
Uptime:               599s

(Refresh dalam 5 detik...)
█
```

4. Test Idempotency

- a. Buat event untuk test dan Publish attempt 1 dengan perintah:

```
event='{
    "topic": "demo",
    "event_id": "demo-idempotent-001",
    "timestamp": "'$(date -u +"%Y-%m-%dT%H:%M:%SZ")'",
    "source": "demo-publisher",
    "payload": { "message": "Testing idempotency" }
}'
```

```
echo -e "\e[33m=== PUBLISH ATTEMPT 1 ===\e[0m"
curl -s -X POST http://localhost:8080/publish \
-H "Content-Type: application/json" \
-d "$event" | python3 -m json.tool
```



```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ # STEP 4.1: Buat event untuk test dan Publish attempt 1
event='{
  "topic": "demo",
  "event_id": "demo-idempotent-001",
  "timestamp": "'$(date -u +"%Y-%m-%dT%H:%M:%SZ")'",
  "source": "demo-publisher",
  "payload": { "message": "Testing idempotency" }
}'

echo -e "\n\e[33m=== PUBLISH ATTEMPT 1 ===\n"
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "$event" | python3 -m json.tool

=== PUBLISH ATTEMPT 1 ===
{
  "detail": [
    {
      "type": "list_type",
      "loc": [
        "body"
      ],
      "msg": "Input should be a valid list",
      "input": {
        "topic": "demo",
        "event_id": "demo-idempotent-001",
        "timestamp": "2025-12-28T07:28:43Z",
        "source": "demo-publisher",
        "payload": {
          "message": "Testing idempotency"
        }
      }
    }
  ],
  "url": "https://errors.pydantic.dev/2.5/v/list_type"
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- b. Publish attempt 2 (same event) dengan perintah:

```

echo -e "\n\e[33m=== PUBLISH ATTEMPT 2 (duplicate) ===\n"
sleep 1
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "$event" | python3 -m json.tool

```

```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\n\e[33m=== PUBLISH ATTEMPT 2 (duplicate) ===\n"
sleep 1
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "$event" | python3 -m json.tool

=== PUBLISH ATTEMPT 2 (duplicate) ===
{
  "detail": [
    {
      "type": "list_type",
      "loc": [
        "body"
      ],
      "msg": "Input should be a valid list",
      "input": {
        "topic": "demo",
        "event_id": "demo-idempotent-001",
        "timestamp": "2025-12-28T07:28:43Z",
        "source": "demo-publisher",
        "payload": {
          "message": "Testing idempotency"
        }
      }
    }
  ],
  "url": "https://errors.pydantic.dev/2.5/v/list_type"
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- c. Publish attempt 3 (same event) dengan perintah:

```

echo -e "\n\e[33m=== PUBLISH ATTEMPT 3 (duplicate) ===\n"
sleep 1
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "$event" | python3 -m json.tool

```



```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\n\e[33m=== PUBLISH ATTEMPT 3 (duplicate) ===\e[0m"
sleep 1
curl -s -X POST http://localhost:8080/publish \
-H "Content-Type: application/json" \
-d "$event" | python3 -m json.tool

=== PUBLISH ATTEMPT 3 (duplicate) ===
{
  "detail": [
    {
      "type": "list_type",
      "loc": [
        "body"
      ],
      "msg": "Input should be a valid list",
      "input": {
        "topic": "demo",
        "event_id": "demo-idempotent-001",
        "timestamp": "2025-12-28T07:28:43Z",
        "source": "demo-publisher",
        "payload": {
          "message": "Testing idempotency"
        }
      },
      "url": "https://errors.pydantic.dev/2.5/v/list_type"
    }
  ]
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

5. Test Idempotency

- Siapkan event untuk concurrent test dengan perintah:

```

EVENT_DATA=$(cat <<EOF
{
  "topic": "concurrent-demo",
  "event_id": "concurrent-001",
  "timestamp": "$(date -u +"%Y-%m-%dT%H:%M:%SZ")",
  "source": "demo",
  "payload": { "test": "concurrency" }
}
EOF
)
echo -e "\e[36m=== CONCURRENT PUBLISH TEST ===\e[0m"
echo "Payload yang disiapkan:"
echo "$EVENT_DATA" | python3 -m json.tool

```

```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ EVENT_DATA=$(cat <<EOF
{
  "topic": "concurrent-demo",
  "event_id": "concurrent-001",
  "timestamp": "$(date -u +"%Y-%m-%dT%H:%M:%SZ")",
  "source": "demo",
  "payload": { "test": "concurrency" }
}
EOF
)

echo -e "\e[36m=== CONCURRENT PUBLISH TEST ===\e[0m"
echo "Payload yang disiapkan:"
echo "$EVENT_DATA" | python3 -m json.tool
=== CONCURRENT PUBLISH TEST ===
Payload yang disiapkan:
{
  "topic": "concurrent-demo",
  "event_id": "concurrent-001",
  "timestamp": "2025-12-28T07:39:42Z",
  "source": "demo",
  "payload": {
    "test": "concurrency"
  }
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- b. Jalankan 5 request bersamaan dengan perintah:

```
echo "Mengirim 5 request ke http://localhost:8080/publish..."
echo ""
PROCESSED_TOTAL=0
for i in {1..5}; do
    RESPONSE=$(curl -s -X POST http://localhost:8080/publish \
        -H "Content-Type: application/json" \
        -d "$EVENT_DATA")

    IS_PROCESSED=$(echo "$RESPONSE" | grep -c '"processed":'
1')
    PROCESSED_TOTAL=$((PROCESSED_TOTAL + IS_PROCESSED))

    echo "Respon Request $i:"
    echo "$RESPONSE" | python3 -m json.tool
    echo "-----"
done
echo -e "\nSTATISTIK AKHIR:"
echo "Total Request      : 5"
echo "Berhasil Process: $PROCESSED_TOTAL"
echo "Terdeteksi Duplikat: $((5 - PROCESSED_TOTAL))"
```

```
    "input": {
        "topic": "concurrent-demo",
        "event_id": "concurrent-001",
        "timestamp": "2025-12-28T07:39:42Z",
        "source": "demo",
        "payload": {
            "test": "concurrency"
        }
    },
    "url": "https://errors.pydantic.dev/2.5/v/list_type"
}
]
}
-----
Respon Request 5:
{
  "detail": [
    {
      "type": "list_type",
      "loc": [
        "body"
      ],
      "msg": "Input should be a valid list",
      "input": {
        "topic": "concurrent-demo",
        "event_id": "concurrent-001",
        "timestamp": "2025-12-28T07:39:42Z",
        "source": "demo",
        "payload": {
          "test": "concurrency"
        }
      },
      "url": "https://errors.pydantic.dev/2.5/v/list_type"
    }
  ]
}
-----
STATISTIK AKHIR:
Total Request      : 5
Berhasil Process: 0
Terdeteksi Duplikat: 5
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

6. Query Events

- a. Query semua event dengan perintah:

```
echo -e "\e[35m=== QUERY EVENTS ===\e[0m"
echo -e "\n[1] GET /events (semua event):"
all_events=$(curl -s http://localhost:8080/events)
echo "$all_events" | python3 -m json.tool
total_count=$(echo "$all_events" | grep -c "event_id")
echo "Estimasi total event (berdasarkan count event_id): $total_count"
$total_count
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\e[35m=== QUERY EVENTS ===\e[0m"
echo -e "\n[1] GET /events (semua event):"
all_events=$(curl -s http://localhost:8080/events)
echo "$all_events" | python3 -m json.tool
total_count=$(echo "$all_events" | grep -c "event_id")
echo "Estimasi total event (berdasarkan count event_id): $total_count"
=== QUERY EVENTS ===

[1] GET /events (semua event):
{
  "count": 0,
  "topic_filter": null,
  "events": []
}
Estimasi total event (berdasarkan count event_id): 0
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

- b. Query events per topic dengan perintah:

```
echo -e "\n[2] GET /events?topic=orders (5 pertama dari topic 'orders'):"
curl -s "http://localhost:8080/events?topic=orders&limit=5" |
python3 -m json.tool
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\n[2] GET /events?topic=orders (5 pertama dari topic 'orders'):"
curl -s "http://localhost:8080/events?topic=orders&limit=5" | python3 -m json.tool

[2] GET /events?topic=orders (5 pertama dari topic 'orders'):
{
  "count": 0,
  "topic_filter": "orders",
  "events": []
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

- c. Query dengan limit dengan perintah:

```
echo -e "\n[3] GET /events?limit=3 (3 events pertama):"
curl -s "http://localhost:8080/events?limit=3" | python3 -m json.tool
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\n[3] GET /events?limit=3 (3 events pertama):"
curl -s "http://localhost:8080/events?limit=3" | python3 -m json.tool

[3] GET /events?limit=3 (3 events pertama):
{
  "count": 0,
  "topic_filter": null,
  "events": []
}
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

7. Persistence Test

- a. Catat statistik sebelum restart dengan perintah:

```

echo -e "\e[33m=== PERSISTENCE TEST ===\e[0m"
echo ""
echo "STEP 1: Mencatat statistik SEBELUM restart"
echo ""
stats_before=$(curl -s http://localhost:8080/stats)
received_before=$(echo $stats_before | python3 -c "import sys,
json; print(json.load(sys.stdin)['received'])")
unique_before=$(echo $stats_before | python3 -c "import sys,
json; print(json.load(sys.stdin)['unique_processed'])")
dropped_before=$(echo $stats_before | python3 -c "import sys,
json; print(json.load(sys.stdin)['duplicate_dropped'])")
echo "Events Received:      $received_before"
echo "Unique Processed:     $unique_before"
echo "Duplicate Dropped:     $dropped_before"
echo ""
echo "Simpan nilai 'Unique Processed': $unique_before"

```

```

ardi_dwi18104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\e[33m=== PERSISTENCE TEST ===\e[0m"
echo ""
echo "STEP 1: Mencatat statistik SEBELUM restart"
echo ""

stats_before=$(curl -s http://localhost:8080/stats)

received_before=$(echo $stats_before | python3 -c "import sys, json; print(json.load(sys.stdin)['received'])")
unique_before=$(echo $stats_before | python3 -c "import sys, json; print(json.load(sys.stdin)['unique_processed'])")
dropped_before=$(echo $stats_before | python3 -c "import sys, json; print(json.load(sys.stdin)['duplicate_dropped'])")

echo "Events Received:      $received_before"
echo "Unique Processed:     $unique_before"
echo "Duplicate Dropped:     $dropped_before"

echo ""
echo "Simpan nilai 'Unique Processed': $unique_before"
=== PERSISTENCE TEST ===

STEP 1: Mencatat statistik SEBELUM restart

Events Received:      2000
Unique Processed:     0
Duplicate Dropped:     0

Simpan nilai 'Unique Processed': 0
ardi_dwi18104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- b. Hentikan sistem dengan perintah:

```

echo ""
echo "STEP 2: Menghentikan Docker Compose..."
docker compose down
echo ""
echo "Volume PostgreSQL tetap ada (persistent)"
sleep 3

```

```

ardi_dwi18104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
echo "STEP 2: Menghentikan Docker Compose..."
docker compose down

STEP 2: Menghentikan Docker Compose...
[+] Running 4/4
✓ Container uas-publisher      Removed      0.1s
✓ Container uas-aggregator     Removed      2.3s
✓ Container uas-storage        Removed      0.7s
✓ Network uassistemterdistribusi_distributed_network Removed      0.3s
ardi_dwi18104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
echo "Volume PostgreSQL tetap ada (persistent)"
sleep 3

Volume PostgreSQL tetap ada (persistent)
ardi_dwi18104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- c. Verifikasi volume persisten dengan perintah:

```
echo ""
echo "STEP 3: Verifikasi volume persisten..."
docker volume ls | grep "uas"
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo "STEP 3: Verifikasi volume persisten..."
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ docker volume ls | grep "uas"

STEP 3: Verifikasi volume persisten...
local      uassistemterdistribusi_aggregator_data
local      uassistemterdistribusi_pg_data
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

- d. Restart sistem dengan perintah:

```
echo ""
echo "STEP 4: Restart Docker Compose..."
docker compose up -d
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo "STEP 4: Restart Docker Compose..."
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ docker compose up -d

STEP 4: Restart Docker Compose...
[*] Running 4/4
✓ Network uassistemterdistribusi_distributed_network Created 0.1s
✓ Container uas-storage Healthy 12.1s
✓ Container uas-aggregator Healthy 22.8s
✓ Container uas-publisher Started 22.9s
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

- e. Verifikasi status layanan dengan perintah:

```
echo ""
echo "STEP 5: Verifikasi status layanan..."
docker compose ps
```

```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo "STEP 5: Verifikasi status layanan..."
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ docker compose ps

STEP 5: Verifikasi status layanan...
NAME                IMAGE                                COMMAND                                SERVICE  CREATED         STATUS
uas-aggregator      uassistemterdistribusi-aggregator  "python -m uvicorn m..."            aggregator 36 seconds ago Up 23 seconds (healthy)
uas-publisher       uassistemterdistribusi-publisher   "python main.py"                     publisher  36 seconds ago Up 12 seconds
uas-storage         postgres:16-alpine                 "docker-entrypoint.s..."            storage    36 seconds ago Up 34 seconds (healthy)
```

- f. Bandingkan statistik setelah restart dengan perintah:

```
echo ""
echo "STEP 6: Membandingkan statistik SETELAH restart..."
stats_after=$(curl -s http://localhost:8080/stats)
received_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['received'])")
unique_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['unique_processed'])")
dropped_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['duplicate_dropped'])")
echo "Events Received:      $received_after"
echo "Unique Processed:     $unique_after"
echo "Duplicate Dropped:     $dropped_after"
```

```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
echo "STEP 6: Membandingkan statistik SETELAH restart..."
stats_after=$(curl -s http://localhost:8080/stats)

received_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['received'])")
unique_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['unique_processed'])")
dropped_after=$(echo $stats_after | python3 -c "import sys, json; print(json.load(sys.stdin)['duplicate_dropped'])")

echo "Events Received:      $received_after"
echo "Unique Processed:     $unique_after"
echo "Duplicate Dropped:    $dropped_after"

STEP 6: Membandingkan statistik SETELAH restart...
Events Received:      3000
Unique Processed:     0
Duplicate Dropped:    0
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- g. Analisis hasil dengan perintah:

```

echo ""
echo "STEP 7: Analisis Hasil"
echo "Sebelum Restart -> Unique Processed: $unique_before"
echo "Setelah Restart -> Unique Processed: $unique_after"

if [ "$unique_before" -eq "$unique_after" ]; then
    echo ""
    echo -e "\e[32m✅ PERSISTENCE TEST PASSED!\e[0m"
    echo "Data berhasil di-recover setelah restart"
else
    echo ""
    echo -e "\e[33m⚠️ Nilai berbeda (publisher mungkin masih aktif atau data hilang)\e[0m"
fi

```

```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo ""
echo "STEP 7: Analisis Hasil"
echo "Sebelum Restart -> Unique Processed: $unique_before"
echo "Setelah Restart -> Unique Processed: $unique_after"

if [ "$unique_before" -eq "$unique_after" ]; then
    echo ""
    echo -e "\e[32m✅ PERSISTENCE TEST PASSED!\e[0m"
    echo "Data berhasil di-recover setelah restart"
else
    echo ""
    echo -e "\e[33m⚠️ Nilai berbeda (publisher mungkin masih aktif atau data hilang)\e[0m"
fi

STEP 7: Analisis Hasil
Sebelum Restart -> Unique Processed: 0
Setelah Restart -> Unique Processed: 0
-bash: !\e[0m: event not found

Data berhasil di-recover setelah restart
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

8. Running Automated Tests

- a. Hentikan docker compose dengan perintah:

```

echo -e "\e[32m=== RUNNING 20 INTEGRATION TESTS ===\e[0m"
docker compose down
sleep 3

```



```

ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\e[32m=== RUNNING 20 INTEGRATION TESTS ===\e[0m"
docker compose down
sleep 3
=== RUNNING 20 INTEGRATION TESTS ===
[+] Running 4/4
  ✓ Container uas-publisher           Removed      0.0s
  ✓ Container uas-aggregator          Removed      0.9s
  ✓ Container uas-storage             Removed      0.5s
  ✓ Network uassistemterdistribusi_distributed_network Removed      0.3s
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

- b. Jalankan test suite dengan perintah:

```

echo ""
echo "Executing pytest..."
echo ""
pytest tests/test_aggregator_mock.py -v --tb=short

```

```

Executing pytest...

===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/d/UAS Sistem Terdistribusi
configfile: pytest.ini
collected 20 items

tests/test_aggregator_mock.py::test_publish_valid_single_event PASSED [ 5%]
tests/test_aggregator_mock.py::test_publish_missing_topic PASSED [ 10%]
tests/test_aggregator_mock.py::test_publish_missing_event_id PASSED [ 15%]
tests/test_aggregator_mock.py::test_publish_invalid_payload PASSED [ 20%]
tests/test_aggregator_mock.py::test_dedup_duplicate_event PASSED [ 25%]
tests/test_aggregator_mock.py::test_dedup_different_topics_same_event_id PASSED [ 30%]
tests/test_aggregator_mock.py::test_dedup_batch_with_duplicates PASSED [ 35%]
tests/test_aggregator_mock.py::test_persistence_data_survives_restart PASSED [ 40%]
tests/test_aggregator_mock.py::test_duplicate_prevented_after_persistent_restart PASSED [ 45%]
tests/test_aggregator_mock.py::test_concurrent_duplicate_publish PASSED [ 50%]
tests/test_aggregator_mock.py::test_concurrent_different_events PASSED [ 55%]
tests/test_aggregator_mock.py::test_stats_consistency_under_load PASSED [ 60%]
tests/test_aggregator_mock.py::test_get_events_empty PASSED [ 65%]
tests/test_aggregator_mock.py::test_get_events_filtered_by_topic PASSED [ 70%]
tests/test_aggregator_mock.py::test_get_events_limit PASSED [ 75%]
tests/test_aggregator_mock.py::test_get_stats_initial PASSED [ 80%]
tests/test_aggregator_mock.py::test_get_stats_after_publish PASSED [ 85%]
tests/test_aggregator_mock.py::test_publish_batch_mixed_valid_invalid PASSED [ 90%]
tests/test_aggregator_mock.py::test_publish_empty_array PASSED [ 95%]
tests/test_aggregator_mock.py::test_publish_not_array PASSED [100%]

===== warnings summary =====
.../usr/lib/python3/dist-packages/_pytest/config/_init_.py:1373
  /usr/lib/python3/dist-packages/_pytest/config/_init_.py:1373: PytestConfigWarning: Unknown config option: asyncio_mode

  self.warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 20 passed, 1 warning in 0.34s =====
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$

```

9. Cleanup

- a. Cleanup dengan perintah:

```

echo -e "\e[31m=== CLEANUP ===\e[0m"
echo ""
echo "Menghentikan semua service..."
docker compose down
echo ""
echo -e "\e[32m✅ Sistem telah dihentikan\e[0m"
echo -e "\e[32m✅ Volume data tetap persisten\e[0m"
echo ""
echo "Untuk melanjutkan: jalankan 'docker compose up -d'"

```



```
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$ echo -e "\e[31m=== CLEANUP ===\e[0m"

echo ""
echo "Menghentikan semua service..."
# Menggunakan 'docker-compose' atau 'docker compose' tergantung versi yang terinstal
docker compose down

echo ""
echo -e "\e[32m✅ Sistem telah dihentikan\e[0m"
echo -e "\e[32m✅ Volume data tetap persisten\e[0m"
echo ""
echo "Untuk melanjutkan: jalankan 'docker compose up -d'"
=== CLEANUP ===

Menghentikan semua service...

✅ Sistem telah dihentikan
✅ Volume data tetap persisten

Untuk melanjutkan: jalankan 'docker compose up -d'
ardi_dwi8104@ArdiDwiSaputra:/mnt/d/UAS Sistem Terdistribusi$
```

Daftar Pustaka

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Distributed systems: Concepts and design (5th ed.). Addison-Wesley.
- Tanenbaum, A. S., & van Steen, M. (2023). Distributed systems (4th ed., Version 4.01). Maarten van Steen.